

VELORA — Infrastructure Architecture

Part 2

Financial, Event-Driven, Queue, Cache, Security, Audit, Scalability

5. Financial Architecture

Double-Entry Bookkeeping

Every financial transaction in VELORA creates a **balanced journal entry** (debit = credit).

Sale Transaction TRX-20260518-00001 (Rp 150.000):

Journal:

Account	Debit	Credit
1-1001 Kas	150.000	
4-1001 Sales		133.784
2-1001 PPN		16.216

(PPN 11% inclusive example)

Auto-Generated Journals

Trigger	Debit Account	Credit Account
Sale (cash)	Kas	Penjualan + PPN
Sale (credit/hutang)	Piutang Pelanggan	Penjualan + PPN
Purchase (cash)	Persediaan	Kas
Purchase (credit)	Persediaan	Hutang Supplier
Expense	Biaya {category}	Kas

Trigger	Debit Account	Credit Account
Refund	Retur Penjualan	Kas
Supplier debt payment	Hutang Supplier	Kas
Customer debt received	Kas	Piutang Pelanggan
Stock adjustment (loss)	Biaya Selisih Stok	Persediaan

Default Chart of Accounts (CoA)

Code	Name	Type
1-1001	Kas	asset
1-1002	Bank	asset
1-2001	Piutang Pelanggan	asset
1-3001	Persediaan Barang	asset
2-1001	Hutang Supplier	liability
2-2001	Hutang Pajak (PPN)	liability
3-1001	Modal Pemilik	equity
3-2001	Laba Ditahan	equity
4-1001	Penjualan	revenue
4-1002	Pendapatan Lain	revenue
5-1001	Harga Pokok Penjualan	cogs
6-1001	Biaya Operasional	expense
6-1002	Biaya Gaji	expense
6-1003	Biaya Sewa	expense
6-1004	Biaya Listrik & Air	expense
6-1005	Biaya Selisih Stok	expense
6-9001	Biaya Lain-lain	expense

Note

Default CoA is auto-seeded when a business is created. Owners can add custom accounts but cannot delete system accounts.

Profit/Loss Calculation

Revenue (4-xxxx)	Rp xxx
– COGS (5-xxxx)	Rp xxx
Gross Profit	Rp xxx
– Expenses (6-xxxx)	Rp xxx
Net Profit	Rp xxx

Calculated on-the-fly from `journal_entries` aggregation by date range and account type. No denormalized profit table — computed from the ledger (source of truth).

Cash Flow Tracking

`cash_flows` table is populated automatically by event listeners. Every money movement creates an entry:

`cash_flows` entries for a day:

Type	Category	Amount	Balance After
in	sales	500.000	1.500.000
in	sales	250.000	1.750.000
out	purchase	300.000	1.450.000
out	expense	50.000	1.400.000
in	debt_rcv	100.000	1.500.000

6. Event-Driven Architecture

Event Catalog

Tenant Events

Event	Dispatched When	Listeners
BusinessCreated	New business registered	SeedDefaultData , CreateDefaultBranch , CreateDefaultWarehouse , SeedChartOfAccounts , CreateTrialSubscription
BusinessLocked	Subscription expired & locked	NotifyOwner , LogSubscriptionAction
BusinessUnlocked	Subscription reactivated	NotifyOwner , LogSubscriptionAction

Subscription Events

Event	Dispatched When	Listeners
SubscriptionCreated	New subscription	LogSubscriptionAction , SendWelcomeEmail
SubscriptionRenewed	Payment received & renewed	LogSubscriptionAction , UpdateBusinessStatus
SubscriptionExpired	Past grace period	LockBusiness , LogSubscriptionAction , NotifyOwner
SubscriptionUpgraded	Changed to higher plan	UpdateFeatureLimits , LogSubscriptionAction
PaymentReceived	Subscription payment confirmed	ActivateSubscription , GenerateInvoice

Sales Events

Event	Dispatched When	Listeners
TransactionCompleted	Sale finalized	DeductStock , CreateCashFlow , CreateAutoJournal , UpdateCustomerStats , EarnLoyaltyPoints , BroadcastToPos
TransactionVoided	Sale voided	RestoreStock , ReverseCashFlow , ReverseJournal , CreateAuditLog
RefundProcessed	Refund completed	RestoreStock (if applicable), CreateCashFlow , CreateAutoJournal , DeductLoyaltyPoints
ShiftOpened	Cashier opens shift	LogActivity , BroadcastToPos
ShiftClosed	Cashier closes shift	CalculateShiftTotals , LogActivity , BroadcastToPos

Inventory Events

Event	Dispatched When	Listeners
StockReceived	Purchase items received	UpdateProductWarehouse , CreateStockMovement , UpdateBatchQuantity , UpdateAvgCost
StockAdjusted	Adjustment approved	UpdateProductWarehouse , CreateStockMovement , CreateAutoJournal
StockOpnameCompleted	Opname approved	ApplyStockDifferences , CreateStockMovements , CreateAutoJournal
StockTransferCompleted	Transfer received	UpdateSourceWarehouse , UpdateDestWarehouse , CreateStockMovements
LowStockDetected	Stock below minimum	SendLowStockAlert
BatchExpiring	Batch near expiry	SendExpiryAlert

Purchasing Events

Event	Dispatched When	Listeners
PurchaseReceived	Goods received	UpdateStock , CreateSupplierDebt (if credit), CreateCashFlow (if cash), CreateAutoJournal
SupplierDebtPaid	Debt payment made	UpdateDebtStatus , CreateCashFlow , CreateAutoJournal

CRM Events

Event	Dispatched When	Listeners
LoyaltyPointsEarned	After eligible sale	UpdateCustomerBalance , CheckTierUpgrade
LoyaltyPointsRedeemed	Points used in sale	UpdateCustomerBalance
CustomerTierUpgraded	Met tier threshold	NotifyCustomer , LogActivity

AI Events

Event	Dispatched When	Listeners
ForecastGenerated	Daily forecast completed	CacheForecastResults , NotifyIfCriticalStock
AnomalyDetected	Anomaly found by detector	StoreAnomaly , SendAnomalyAlert (if high severity)
AIQueryCompleted	AI chat/query answered	LogAIUsage , UpdateUsageQuota
AIQuotaExceeded	Business hits AI query limit	NotifyOwner , BlockFurtherQueries

Payment Events (Midtrans)

Event	Dispatched When	Listeners
MidtransWebhookReceived	Webhook POST received	VerifySignature , ProcessPaymentStatus

Event	Dispatched When	Listeners
PaymentSettled	Midtrans status = settlement	ActivateSubscription , CreateInvoice , SendReceipt
PaymentExpired	Midtrans status = expire	MarkPaymentFailed , SendPaymentReminder
ManualPaymentUploaded	User uploads transfer proof	NotifyPlatformAdmin
ManualPaymentVerified	Admin verifies transfer	ActivateSubscription , SendReceipt , LogAction

Event Processing Strategy

Synchronous Events (must complete before response):

- DeductStock (critical for POS accuracy)
- CreateStockMovement

Queued Events (async via listener implements ShouldQueue):

- SendLowStockAlert
- SendExpiryAlert
- NotifyOwner
- GenerateInvoice
- CreateAutoJournal
- UpdateCustomerStats
- EarnLoyaltyPoints

7. Queue Strategy

Queue Configuration (Redis Driver)

Queue Name	Priority	Purpose	Workers
critical	Highest	Payment processing, stock deduction (if async)	3
default	Normal	Journals, cash flows, loyalty, customer stats	2

Queue Name	Priority	Purpose	Workers
notifications	Low	Email, push, SMS	1
reports	Low	Report generation, exports	1
maintenance	Lowest	Cleanup, log rotation, expiry checks	1

Worker Command

```
php artisan queue:work redis --queue=critical,default,notifications,reports,maintenance
```

Key Jobs

Job	Queue	Timeout	Retries	Backoff
ProcessPayment	critical	30s	3	10s, 30s, 60s
CreateAutoJournal	default	15s	3	5s
UpdateCustomerStats	default	10s	3	5s
SendTransactionReceipt	notifications	30s	3	15s
SendLowStockAlert	notifications	30s	2	30s
GenerateSalesReport	reports	120s	2	60s
ExportToExcel	reports	300s	1	—
CheckExpiredSubscriptions	maintenance	60s	1	—
CleanOldActivityLogs	maintenance	120s	1	—
GenerateDailyForecast	reports	120s	2	60s
GenerateRestockSuggestions	reports	90s	2	60s
DetectAnomalies	reports	120s	1	—
ProcessAIChatQuery	default	30s	2	5s
ProcessMidtransWebhook	critical	15s	3	5s, 15s, 30s
SendPaymentReminder	notifications	30s	2	30s

Failed Job Handling

1. Failed jobs stored in `failed_jobs` table
2. Critical failures → immediate Sentry alert
3. Daily scheduled check: `php artisan queue:retry --queue=critical`
4. Failed payment jobs → mark payment as `failed`, notify business owner

Rate Limiting (Queue)

- Max 100 notification jobs per business per hour
- Max 5 report generation jobs per business per hour
- Max 1000 stock movement jobs per business per hour

8. Cache Strategy

Cache Driver: Redis with Tagged Cache

Cache Architecture:

Redis Instance
Tag: business:{uuid}
├─ products:list → cached 5 min
├─ products:{id} → cached 10 min
├─ categories:tree → cached 30 min
├─ settings:all → cached 60 min
├─ subscription:current → cached 5 min
└─ feature_limits → cached 60 min
Tag: branch:{uuid}
├─ active_shift → cached 1 min
└─ payment_methods → cached 30 min
Global (no tag):
├─ subscription_plans → cached 1 hour
├─ system_units → cached 24 hours
├─ permissions:all → cached 1 hour
└─ currencies:all → cached 24 hours
└─ exchange_rates → cached 12 hours
Tag: ai:{business_uuid}
├─ forecast:latest → cached 1 hour
├─ restock:suggestions → cached 3 hours
├─ segments:customer → cached 6 hours
└─ chat:{query_hash} → cached 5 min

Cache Keys Convention

Pattern: {scope}:{entity}:{identifier}:{sub}

Examples:

business:{uuid}:products:list:page_1
business:{uuid}:products:{product_id}
business:{uuid}:categories:tree
business:{uuid}:settings:pos
business:{uuid}:subscription:current
branch:{uuid}:shift:active
global:plans:all
global:units:all

Cache Invalidation Rules

Event	Invalidate
Product created/updated/deleted	business:{id}:products:*
Category changed	business:{id}:categories:*
Settings changed	business:{id}:settings:*
Subscription changed	business:{id}:subscription:* , business:{id}:feature_limits
Stock movement	business:{id}:products:{product_id} (stock fields only)
Shift opened/closed	branch:{id}:shift:*
Plan changed (admin)	global:plans:*
Currency/exchange rate updated	global:currencies:* , global:exchange_rates
AI forecast generated	ai:{id}:forecast:*
AI restock updated	ai:{id}:restock:*
AI query (same hash)	Already cached by query hash

Cache-Aside Pattern (Implementation)

Service layer:

- 1. Check cache for key
- 2. If hit → return cached data
- 3. If miss → query DB → store in cache → return
- 4. On write → invalidate related cache tags

9. Security Strategy

Authentication

Layer	Implementation
Token Auth	Laravel Sanctum (SPA + Mobile tokens)
Token Expiry	Access token: 24h, Refresh: 30 days
Password Hash	bcrypt (12 rounds)
Login Throttle	5 attempts / minute per email+IP
2FA	Optional TOTP (Google Authenticator) for owner/admin
Session	Redis-backed, per-device tracking

API Security

Concern	Solution
Rate Limiting	60 req/min (default), 120 req/min (Pro+), 300 req/min (Enterprise)
CORS	Whitelist specific frontend domains
HTTPS	Enforce TLS 1.2+ everywhere
Input Validation	FormRequest on every endpoint, no raw input
SQL Injection	Eloquent ORM (parameterized queries), no raw DB::raw without bindings
XSS	JSON-only API (no HTML rendering), sanitize stored text

Concern	Solution
CSRF	Not applicable (stateless API with Sanctum tokens)
Mass Assignment	<code>\$fillable</code> whitelist on all models

Data Security

Concern	Solution
Sensitive Data	Encrypt: <code>tax_id</code> , <code>account_number</code> (at-rest encryption via Laravel <code>Crypt</code>)
PII	Email, phone, address — accessible only via authorized endpoints
API Keys	Hashed storage (like passwords), shown once on creation
File Upload	Validate MIME type + extension, max size 5MB, virus scan (optional)
Backup	Daily automated MySQL dumps, encrypted, stored off-site

Tenant Security

Concern	Solution
Data Isolation	Global scope <code>business_id</code> — no cross-tenant leaks
IDOR Prevention	Always scope queries: <code>Product::where('business_id', \$businessId)->findOrFail(\$id)</code> — never <code>Product::findOrFail(\$id)</code>
Privilege Escalation	Role level checks: user cannot assign role higher than their own
Branch Access	<code>branch_user</code> pivot checked via middleware

Security Headers

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'
Referrer-Policy: strict-origin-when-cross-origin
```

10. Audit Log Strategy

What Gets Audited

Level	What	Storage
Activity Log	User actions: login, logout, page views, button clicks	activity_logs table
Audit Log	Data changes: create, update, delete with before/after values	audit_logs table
Stock Movement	All inventory changes	stock_movements table (dedicated)
Financial Ledger	All money movements	cash_flows + journal_entries (dedicated)
Subscription Log	Plan changes, payments, expirations	subscription_logs table (dedicated)

Audit Log Implementation

Auditable Trait — added to models that need audit tracking:

Models with Auditable trait:

- Product, ProductVariant
- Transaction (status changes only)
- Purchase
- StockAdjustment, StockOpname
- Expense, Income
- Customer
- Supplier
- Discount, Voucher
- BusinessSetting
- Role (custom roles)

What's captured per audit entry:

- auditable_type + auditable_id (polymorphic)
- event : created, updated, deleted, restored
- old_values : JSON snapshot of changed fields (before)
- new_values : JSON snapshot of changed fields (after)
- user_id : who made the change
- ip_address : from where
- url : which endpoint

Retention Policy

Log Type	Retention	Cleanup
Activity Logs	90 days	Daily scheduled cleanup
Audit Logs	1 year	Monthly cleanup
Stock Movements	Forever	Never delete (legal requirement)
Financial Journals	Forever	Never delete (legal requirement)
Subscription Logs	2 years	Yearly cleanup

Cleanup Job

Schedule: Daily at 2:00 AM

Job: CleanOldLogs

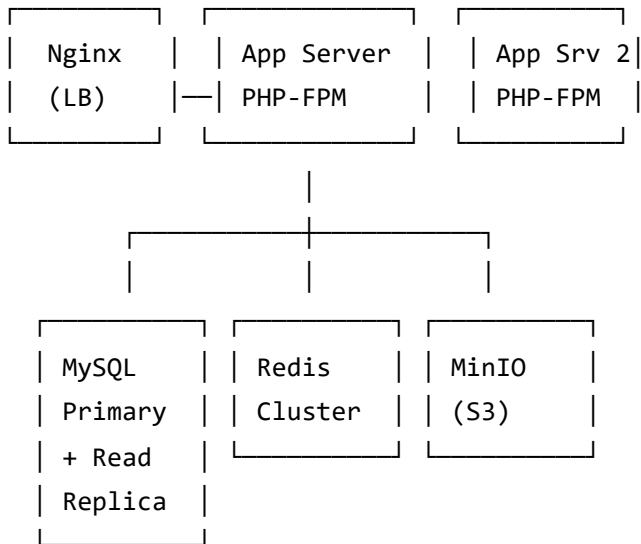
- Delete activity_logs older than 90 days
- Delete audit_logs older than 365 days
- Archive to cold storage if needed (future)

11. Scalability Strategy

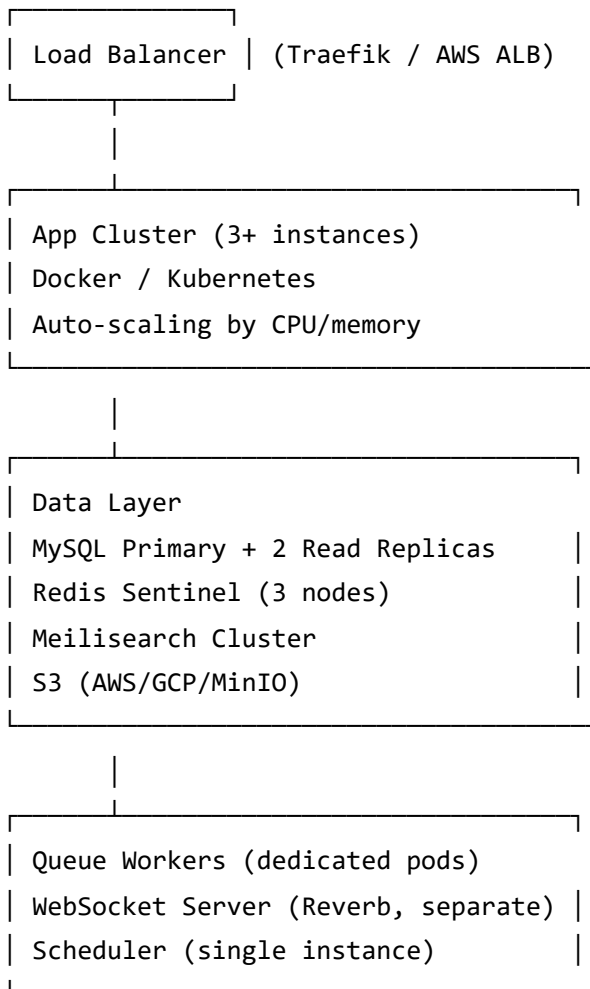
Phase 1: Single Server (0–1,000 businesses)

Single VPS / Cloud VM
Nginx
PHP-FPM (Laravel)
MySQL 8.0
Redis 7
Queue Workers
Laravel Reverb (WS)

Phase 2: Separated Services (1,000–10,000 businesses)



Phase 3: Full Scale (10,000+ businesses)



Database Optimization

Technique	Implementation
Indexing	Composite index on (business_id, ...) for every tenant query
Read Replicas	Route SELECT queries to replica via Laravel's readWriteConnection
Query Optimization	Eager loading (with()), no N+1 queries, select() specific columns
Partitioning	stock_movements and audit_logs partitioned by month (RANGE on created_at)
Archiving	Old transactions (>2 years) → archive tables or cold storage
Connection Pooling	PgBouncer-equivalent or persistent connections

Horizontal Scaling Rules

Component	Scale Strategy
App Servers	Stateless — add more behind load balancer
Queue Workers	Scale by queue depth monitoring
WebSocket	Sticky sessions or Redis pub/sub for cross-server broadcasts
Database	Vertical first → Read replicas → Sharding by business_id (last resort)
Cache (Redis)	Redis Sentinel → Redis Cluster
Search	Meilisearch cluster with replicas
File Storage	S3 (infinitely scalable)

Performance Targets

Metric	Target
API Response (p95)	< 200ms
POS Transaction Complete	< 500ms

Metric	Target
Search Query	< 100ms
Report Generation	< 30s (simple), < 5min (complex)
WebSocket Latency	< 100ms
Concurrent POS per branch	10+ simultaneous cashiers
Uptime SLA	99.9%

Summary of All Architecture Documents

Document	Content
[Implementation Plan] (file:///C:/Users/YP/.gemini/antigravity/brain/7880df49-c88e-4c8f-82bc-eb5a42cebb61/implementation_plan.md)	System arch, modules, folder structure, API standard, conventions, service layer
[DB Part 1](file:///C:/Users/YP/.gemini/antigravity/brain/7880df49-c88e-4c8f-82bc-eb5a42cebb61/database_architecture_part1.md)	Core SaaS, Subscription, Auth, POS tables
[DB Part 2](file:///C:/Users/YP/.gemini/antigravity/brain/7880df49-c88e-4c8f-82bc-eb5a42cebb61/database_architecture_part2.md)	Inventory, Sales, Purchasing tables
[DB Part 3](file:///C:/Users/YP/.gemini/antigravity/brain/7880df49-c88e-4c8f-82bc-eb5a42cebb61/database_architecture_part3.md)	CRM, Finance, Settings tables
[Infra Part 1](file:///C:/Users/YP/.gemini/antigravity/brain/7880df49-c88e-4c8f-82bc-eb5a42cebb61/infrastructure_architecture.md)	Multi-tenant, Subscription lifecycle, Inventory, Roles
[Infra Part 2](file:///C:/Users/YP/.gemini/antigravity/brain/7880df49-c88e-4c8f-82bc-eb5a42cebb61/infrastructure_architecture_part2.md)	Financial, Events, Queue, Cache, Security, Audit, Scalability

Seluruh architecture planning VELORA telah selesai. Silakan review dan berikan feedback sebelum masuk ke fase coding.